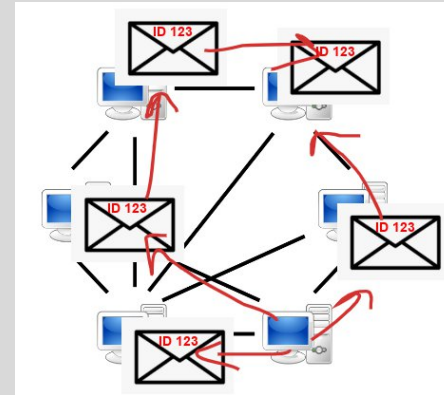
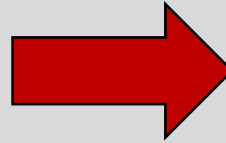
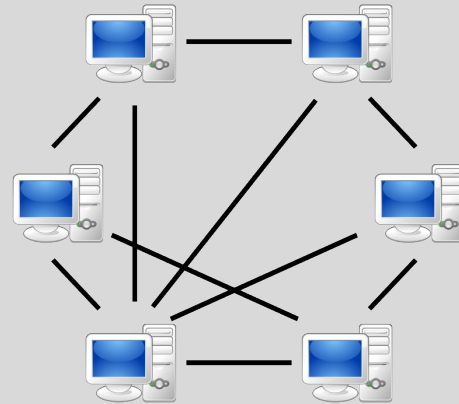




Informatik 2: Threads





Heute

C++

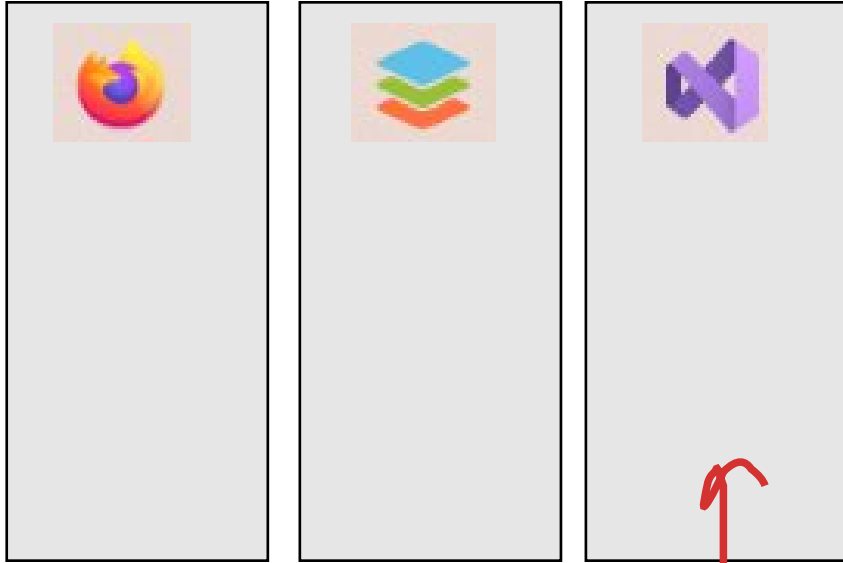
- Threads



Im Betriebssystem laufen verschiedene parallele Programme, auch Prozesse genannt, die jeweils einen eigenen Adressbereich haben.

Zum Beispiel können ein Office-Programm und der Internet-Browser nebeneinander laufen.

Prozesse



Im Betriebssystem laufen verschiedene parallele Programme, auch Prozesse genannt, die jeweils einen eigenen Adressbereich haben.

Zum Beispiel können ein Office-Programm und der Internet-Browser nebeneinander laufen.



Im Betriebssystem laufen verschiedene parallele Programme, auch Prozesse genannt, die jeweils einen eigenen Adressbereich haben.

Zum Beispiel können ein Office-Programm und der Internet-Browser nebeneinander laufen.

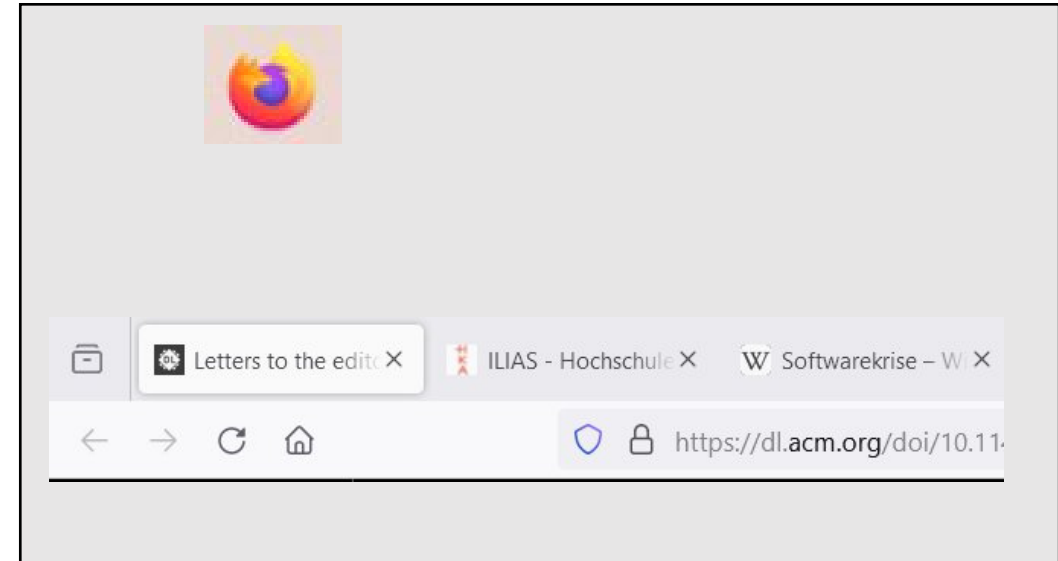
} Multitasking

Prozesse & Threads



Im Betriebssystem laufen verschiedene parallele Programme, auch Prozesse genannt, die jeweils einen eigenen Adressbereich haben.

Zum Beispiel können ein Office-Programm und der Internet-Browser nebeneinander laufen.



Im Unterschied zu Prozessen teilen sich Threads (thread: englisch für »Faden«) einen Adressraum.

Zu einem Prozess gehörende Threads können also auf dieselben Variablen zugreifen.

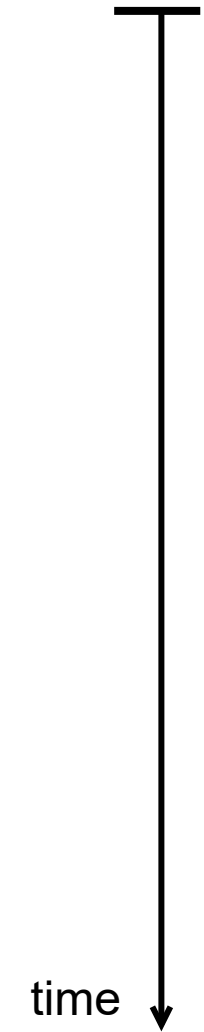


Threads

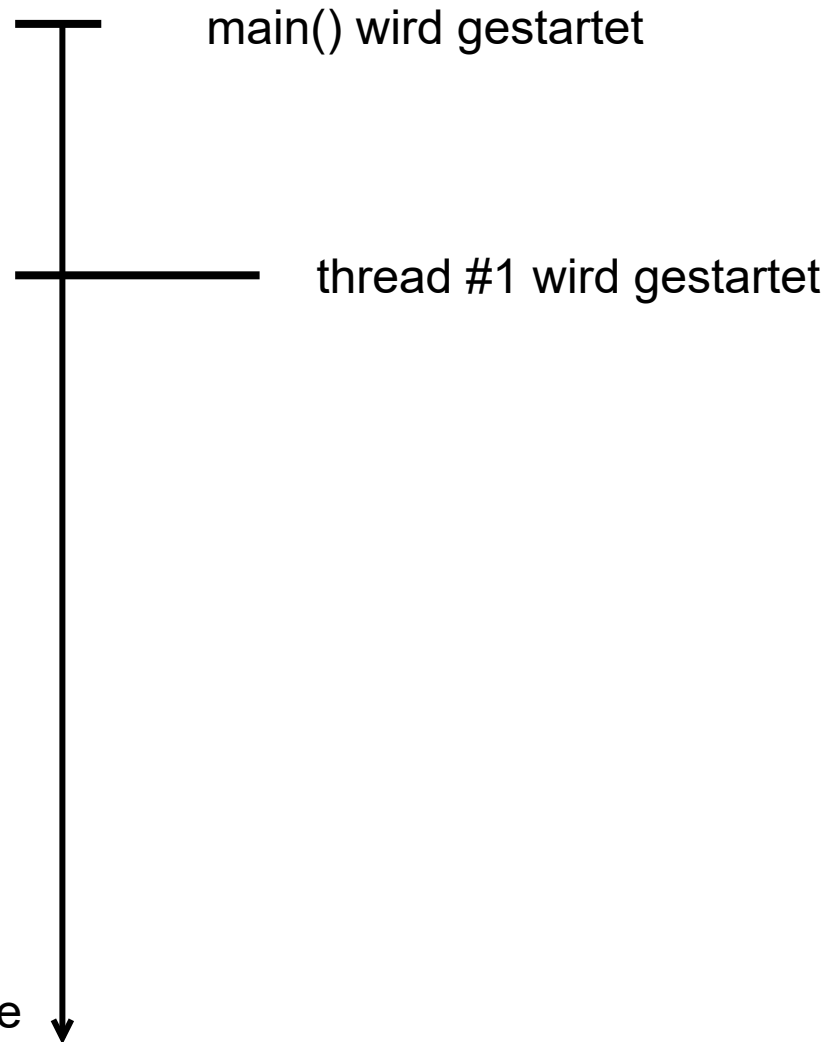


main() wird gestartet

Innerhalb eines main-Programms



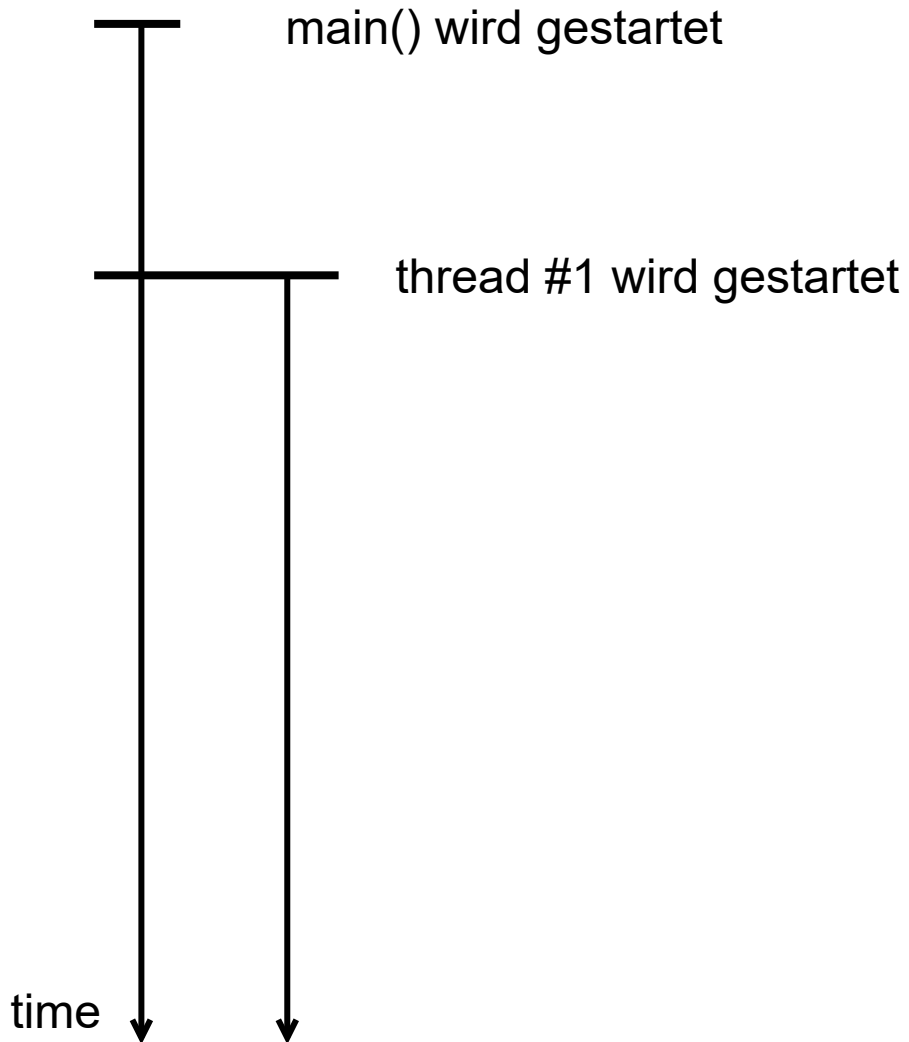
Threads



Innerhalb eines main-Programms (das selbst einen Thread darstellt), kann ein weiterer Thread gestartet werden.



Threads

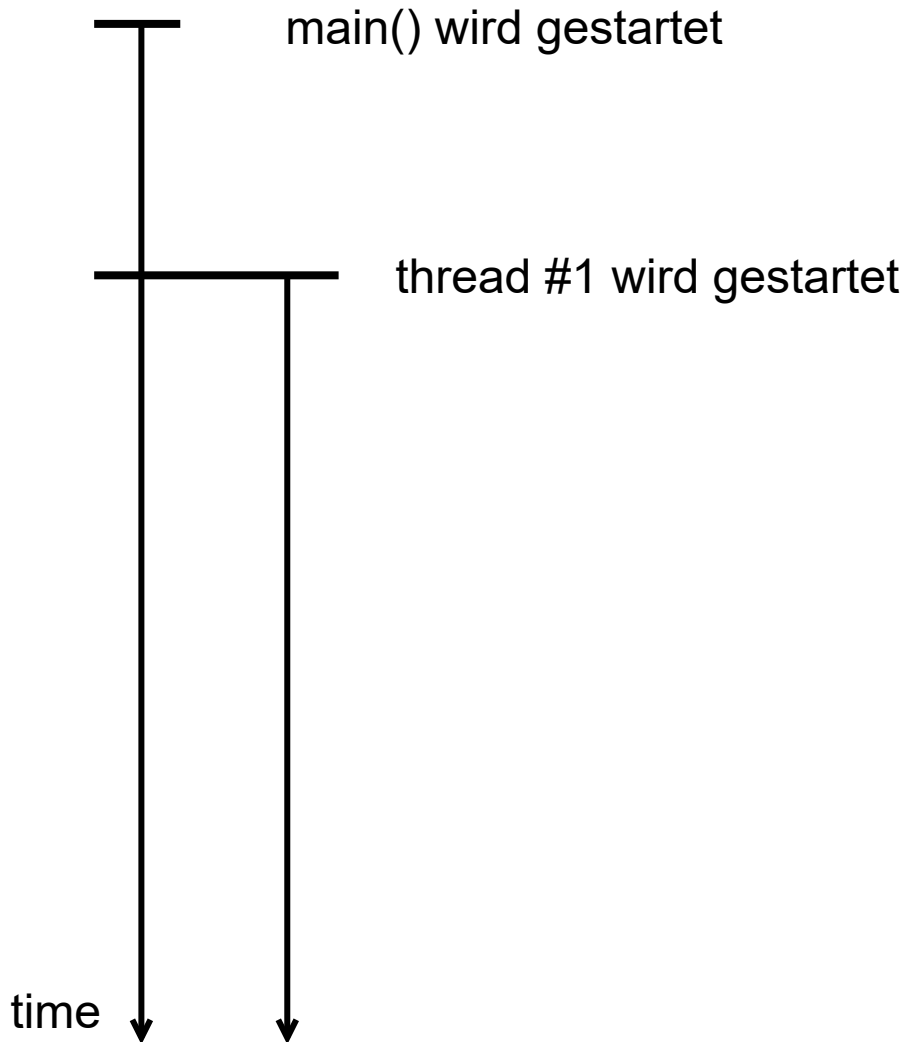


Innerhalb eines main-Programms (das selbst einen Thread darstellt), kann ein weiterer Thread gestartet werden.

Das bedeutet, dass ab diesem Zeitpunkt ein weiterer Faden der Programmausführung neben dem erzeugenden Faden herläuft.



Threads



Innerhalb eines main-Programms (das selbst einen Thread darstellt), kann ein weiterer Thread gestartet werden.

Das bedeutet, dass ab diesem Zeitpunkt ein weiterer Faden der Programmausführung neben dem erzeugenden Faden herläuft.

Wenn es gleich viele oder mehr CPUs als parallele Prozesse/Threads gibt, können die Prozesse/Threads **echt parallel** abgearbeitet werden.

Andernfall spricht man von **quasi-parallel**. Diese nur scheinbare Parallelität wird bei wenigen oder nur einer CPU erzeugt, indem in kurzen Zeitabständen jeder Thread mal zur Ausführung kommt. Das Betriebssystem übernimmt die Aufteilung der Prozesse auf die CPUs. Dieser Vorgang heißt Einplanung (englisch scheduling).

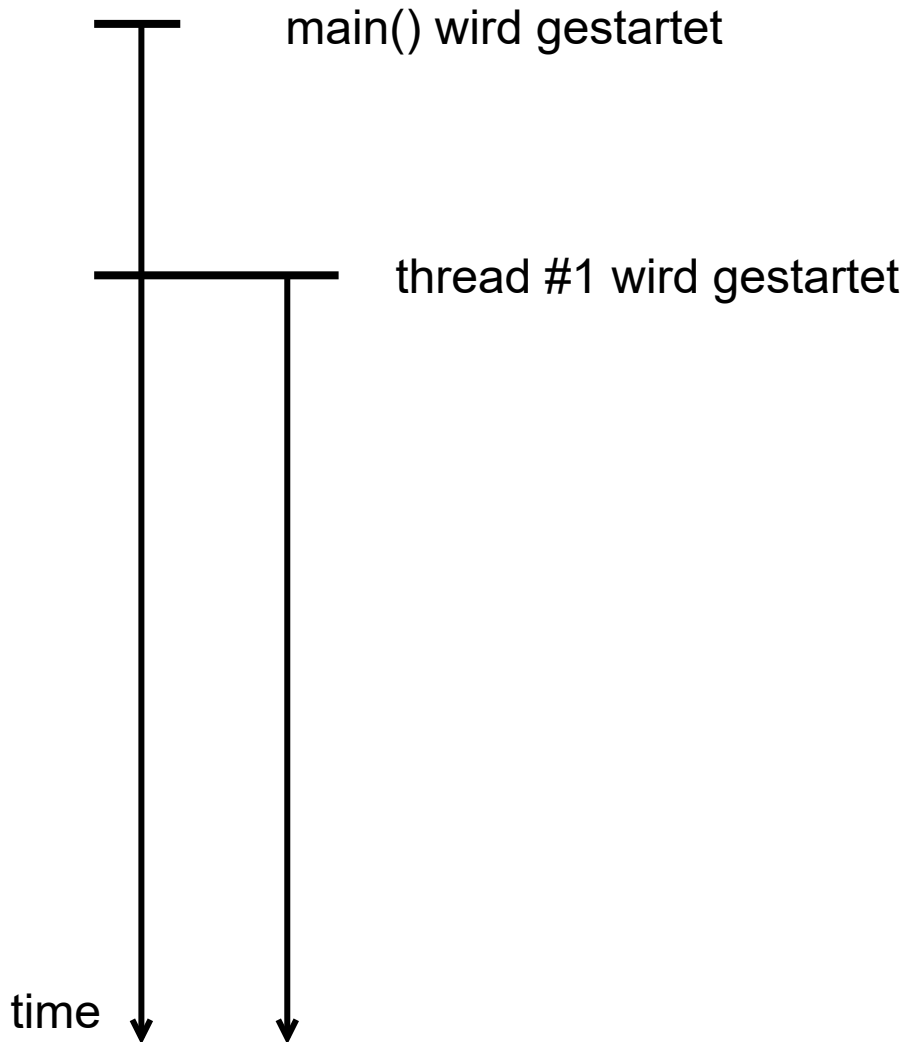




C++



Threads



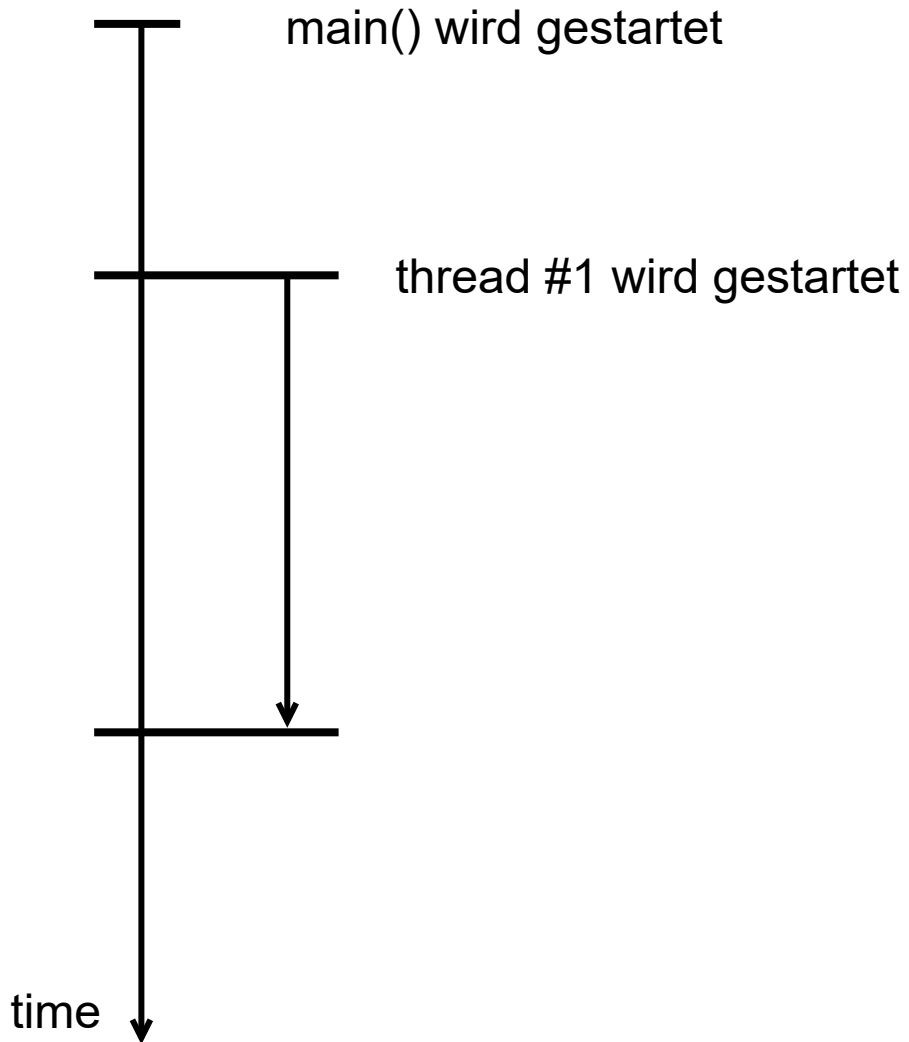
```
#include <iostream>
#include <thread>
#include <windows.h> // for Sleep() only
using namespace std;

void IchBinEineErsteFunktion() {
    cout << "Ich bin eine erste Funktion\n";
}

int main()
{
    cout << "Ich bin der Main thread\n";
    thread t1(IchBinEineErsteFunktion); // thread #1

    Sleep (100);
    return 0;
}
```

Auf Threadende warten



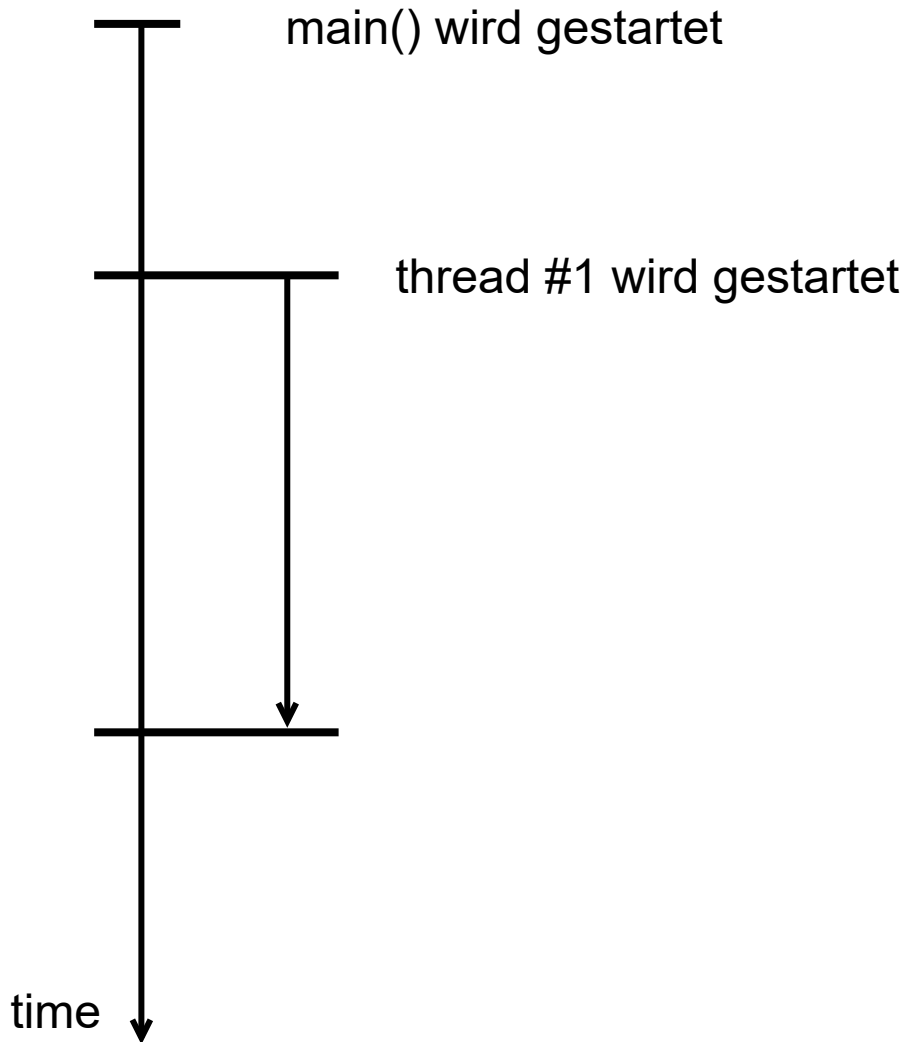
Innerhalb eines main-Programms (das selbst einen Thread darstellt), kann ein weiterer Thread gestartet werden.

Das bedeutet, dass ab diesem Zeitpunkt ein weiterer Faden der Programmausführung neben dem erzeugenden Faden herläuft.

Das main-Programm kann auf das Ende eines Threads warten.



Auf Threadende warten



```
#include <iostream>
#include <thread>

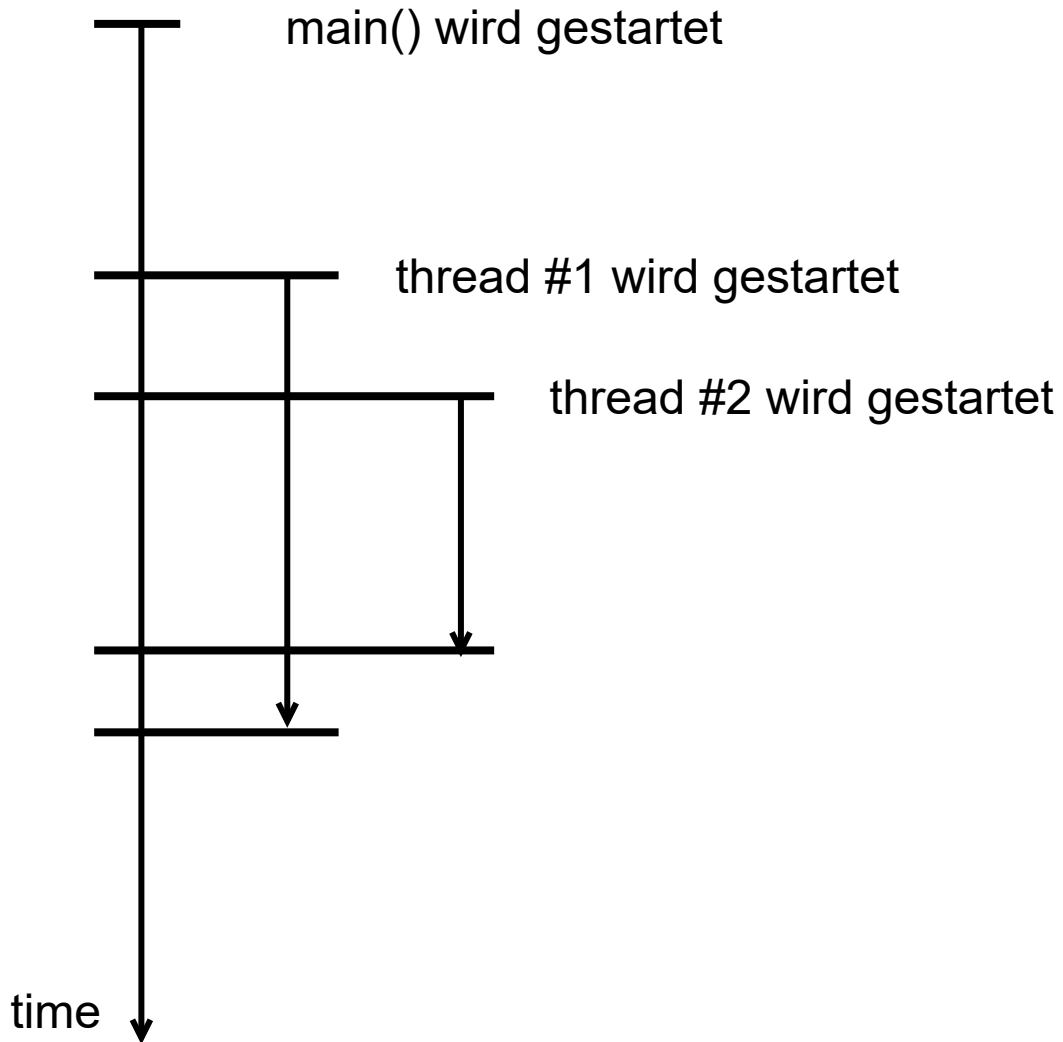
using namespace std;

void IchBinEineErsteFunktion() {
    cout << "Ich bin eine erste Funktion\n";
}

int main()
{
    cout << "Ich bin der Main thread\n";
    thread t1(IchBinEineErsteFunktion); // thread #1

    t1.join();
    return 0;
}
```

Multithreading...

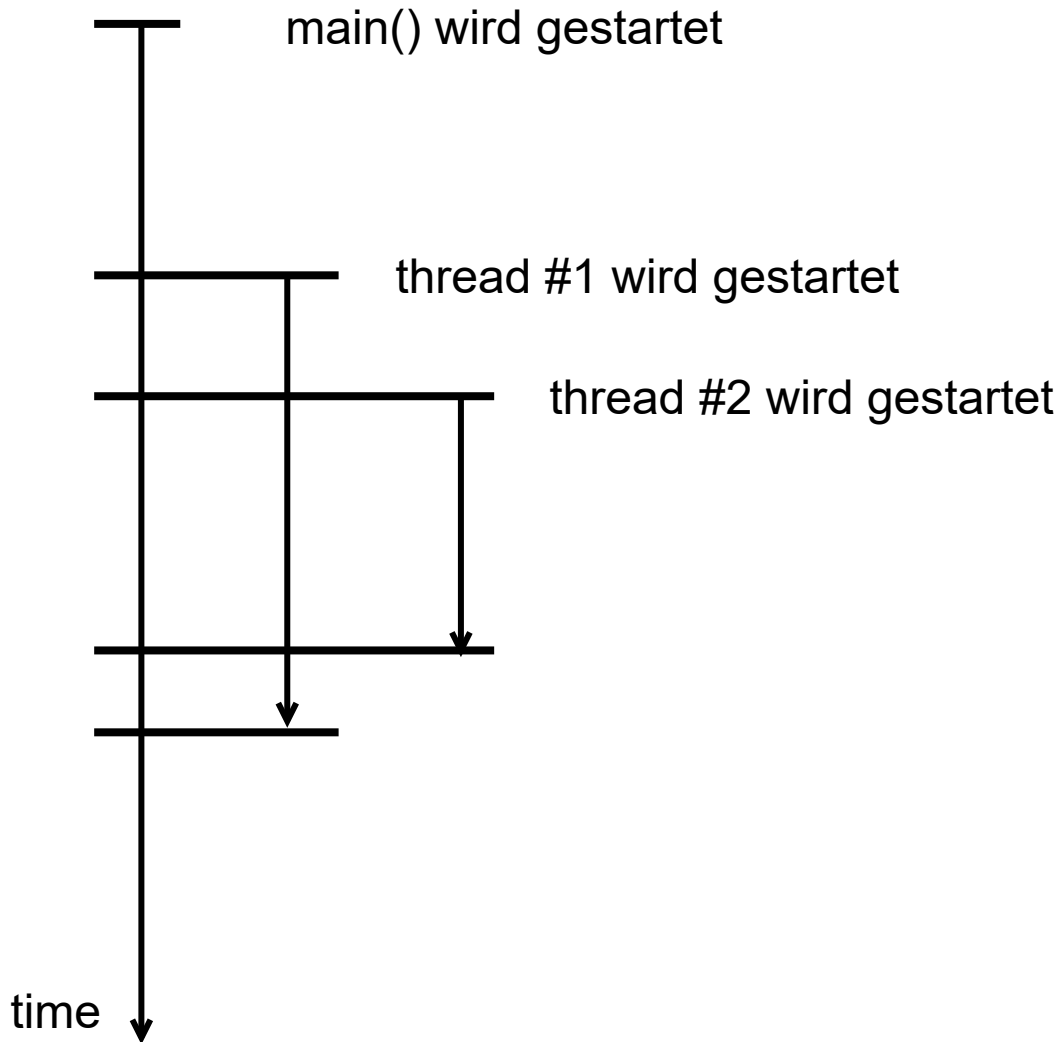


Innerhalb eines main-Programms (das selbst einen Thread darstellt), können mehrere weitere Thread gestartet werden.

Das bedeutet, dass ab diesem Zeitpunkt weitere Fäden der Programmausführung neben dem erzeugenden Faden herläuft.

Das main-Programm kann auf das Ende der Threads warten.

Multithreading...



```
#include <iostream>
#include <thread>

using namespace std;

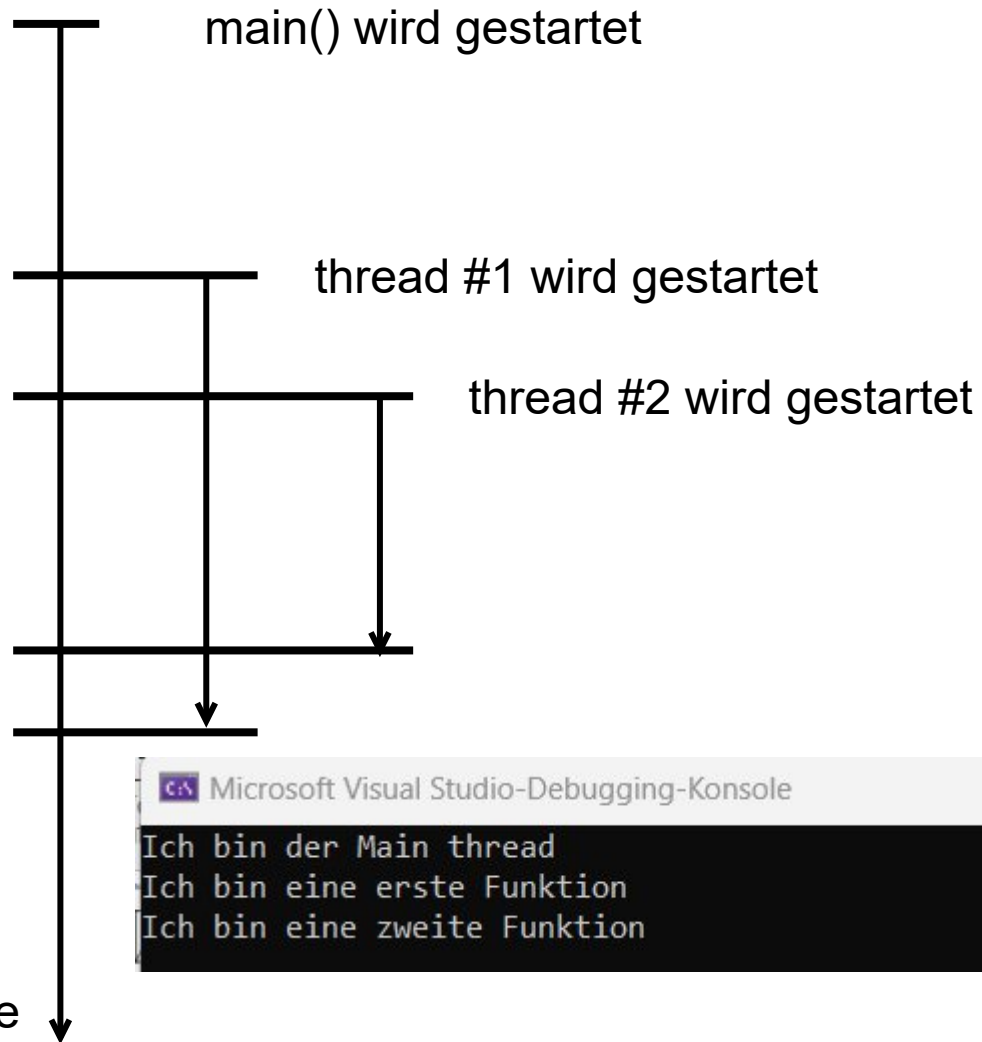
void IchBinEineErsteFunktion() {
    cout << "Ich bin eine erste Funktion\n";
}

void IchBinEineZweiteFunktion() {
    cout << "Ich bin eine zweite Funktion\n";
}

int main()
{
    cout << "Ich bin der Main thread\n";
    thread t1(IchBinEineErsteFunktion); // thread #1
    thread t2(IchBinEineZweiteFunktion); // thread #2

    t1.join();
    t2.join();
    return 0;
}
```


Multithreading...



```
#include <iostream>
#include <thread>

using namespace std;

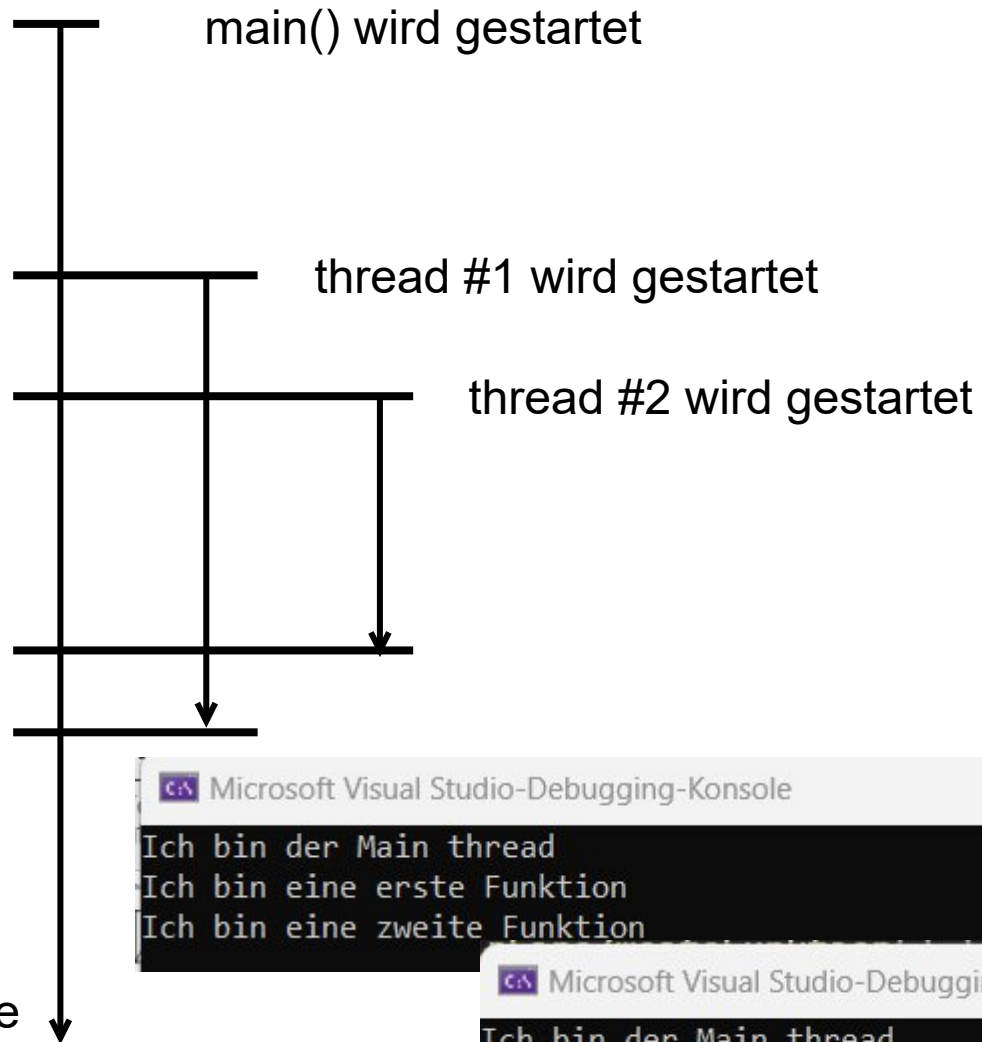
void IchBinEineErsteFunktion() {
    cout << "Ich bin eine erste Funktion\n";
}

void IchBinEineZweiteFunktion() {
    cout << "Ich bin eine zweite Funktion\n";
}

int main()
{
    cout << "Ich bin der Main thread\n";
    thread t1(IchBinEineErsteFunktion); // thread #1
    thread t2(IchBinEineZweiteFunktion); // thread #2

    t1.join();
    t2.join();
    return 0;
}
```

Multithreading...



Microsoft Visual Studio-Debugging-Konsole

```
Ich bin der Main thread  
Ich bin eine erste Funktion  
Ich bin eine zweite Funktion
```

Microsoft Visual Studio-Debugging-Konsole

```
Ich bin der Main thread  
Ich bin eine zweite Funktion  
Ich bin eine erste Funktion
```

```
#include <iostream>
#include <thread>

using namespace std;

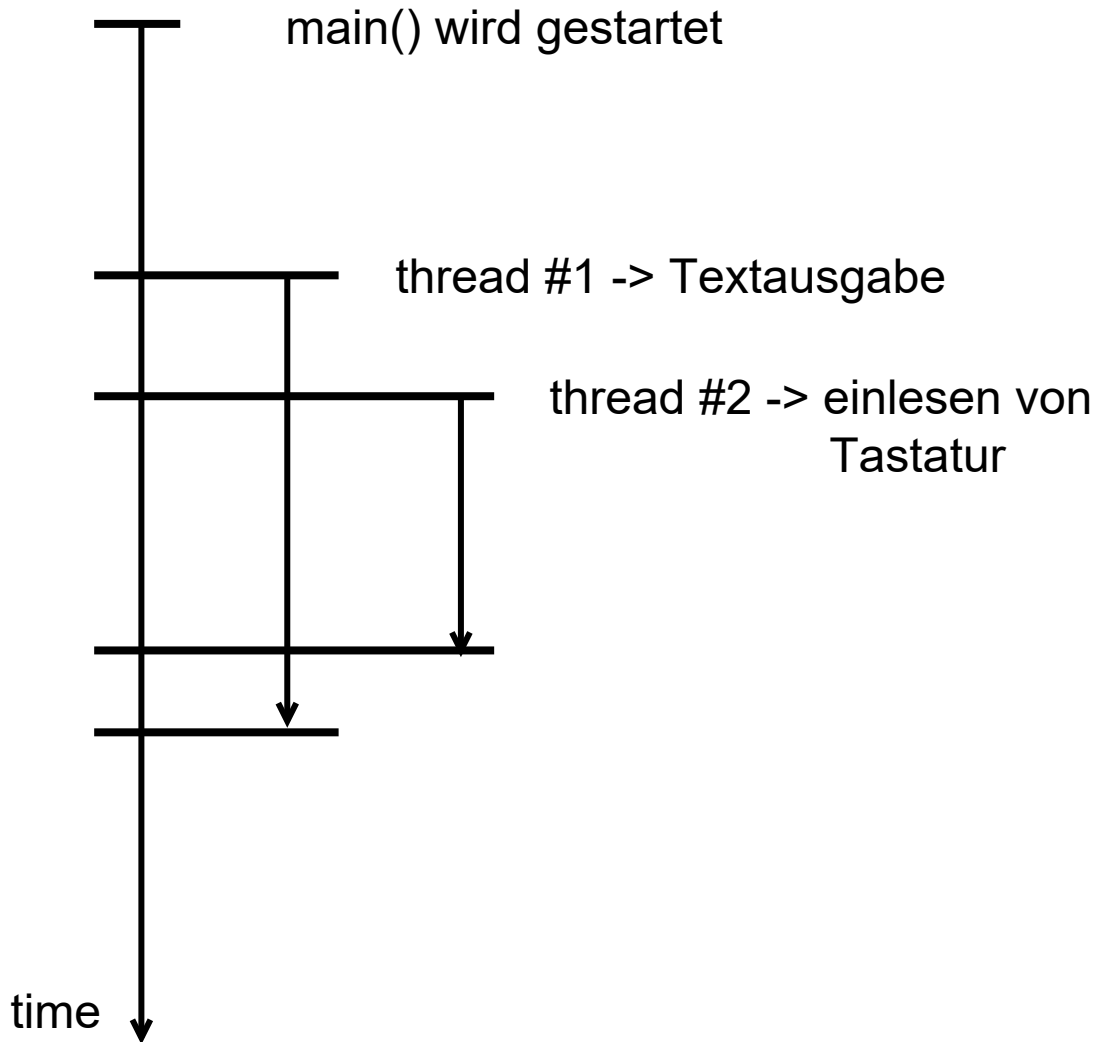
void IchBinEineErsteFunktion() {
    cout << "Ich bin eine erste Funktion\n";
}

void IchBinEineZweiteFunktion() {
    cout << "Ich bin eine zweite Funktion\n";
}

int main()
{
    cout << "Ich bin der Main thread\n";
    thread t1(IchBinEineErsteFunktion); // thread #1
    thread t2(IchBinEineZweiteFunktion); // thread #2

    t1.join();
    t2.join();
    return 0;
}
```

Mal ein nützliches Beispiel...



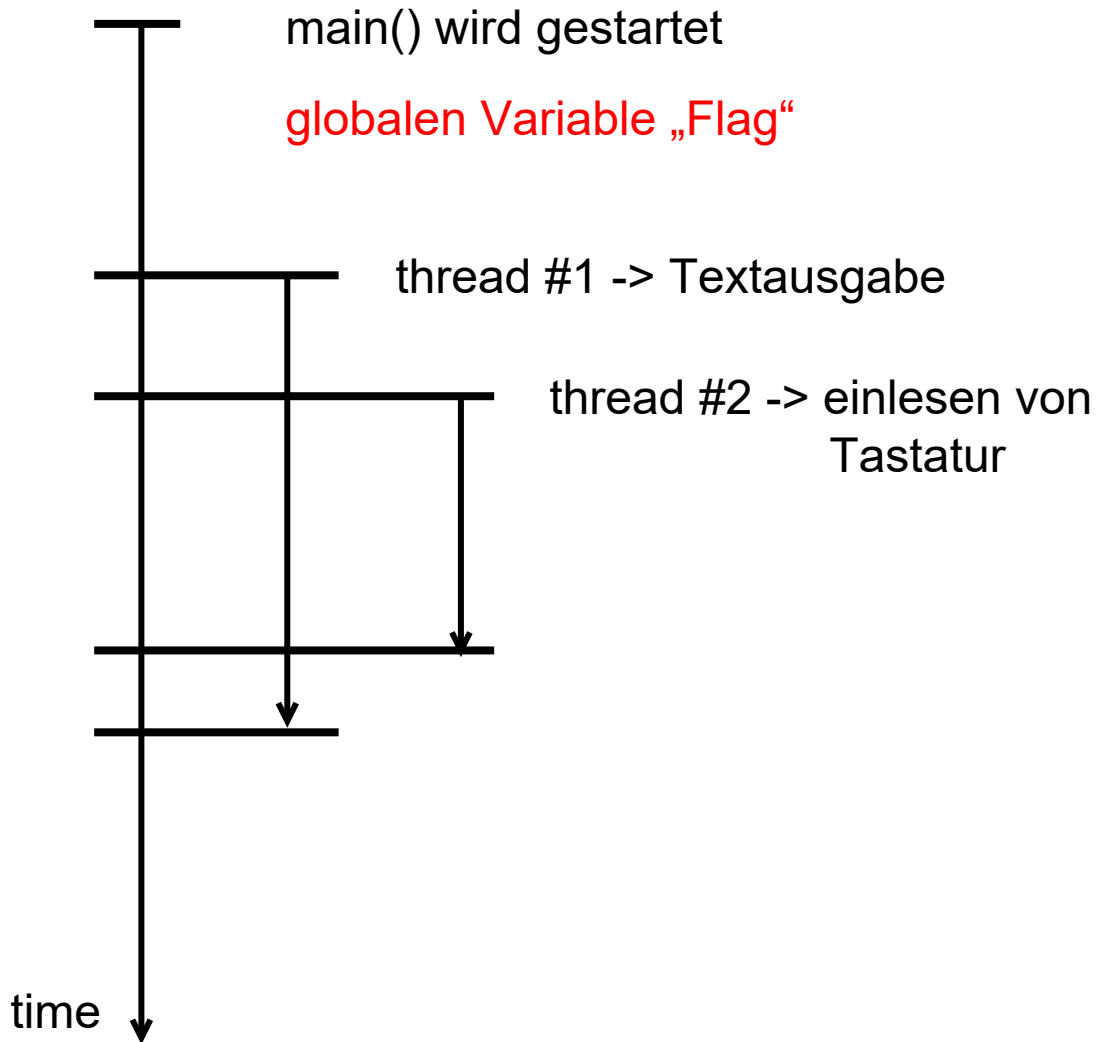
```
#include <iostream>
#include <thread>
#include <windows.h>
using namespace std;

void TextEingabe() {
    char Eingabe;
    cout << "Dieser Thread wartete nur auf Eingabe...\n";
    cin >> Eingabe;
}

void SomeTextOutput() {
    for (int z = 1; z < 1000; z++) {
        cout << z << "\tDieserThread erzeugt einfach Textausgabe\n";
        Sleep(1);
    }
}

int main()
{
    thread t1(TextEingabe); // thread #1
    thread t2(SomeTextOutput); // thread #2
    t1.join();
    t2.join();
    return 0;
}
```

Mal ein nützliches Beispiel...



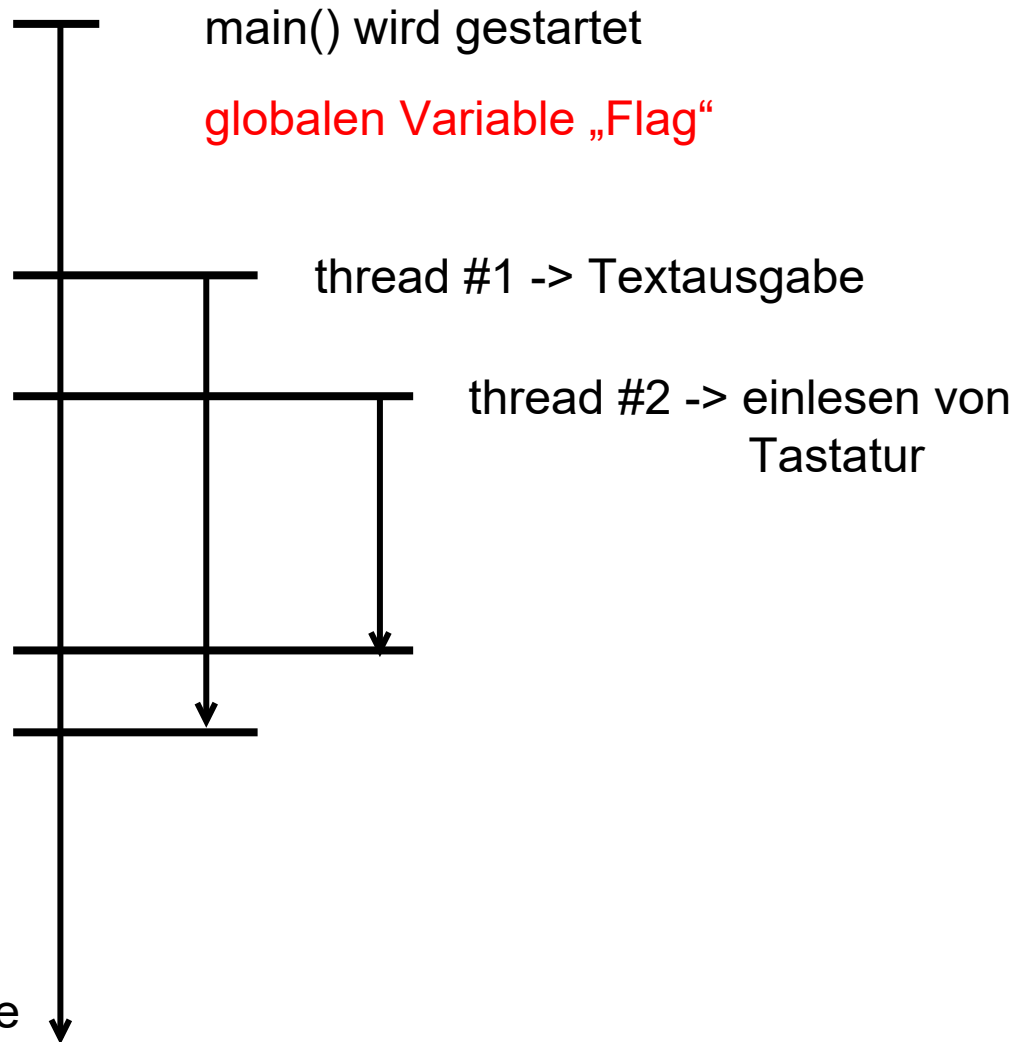
```
#include <iostream>
#include <thread>
#include <windows.h>
using namespace std;
int Flag = 0;

void TextEingabe() {
    char Eingabe;
    cout << "Dieser Thread wartete nur auf Eingabe... s für Ende\n";
    cin >> Eingabe;
    if (Eingabe == 's' || Eingabe == 'S') Flag = 1;
}

void SomeTextOutput() {
    for (int z = 1; z < 1000; z++) {
        cout << z << "\tDieserThread erzeugt einfach Textausgabe\n";
        if (Flag == 0) Sleep(1); else break;
    }
}

int main() {
    thread t1(TextEingabe); // thread #1
    thread t2(SomeTextOutput); // thread #2
    t1.join(); t2.join(); return 0;}
```

Mal ein nützliches Beispiel...



Nutzen einer globalen Variablen, um Informationen zwischen Thread auszutauschen



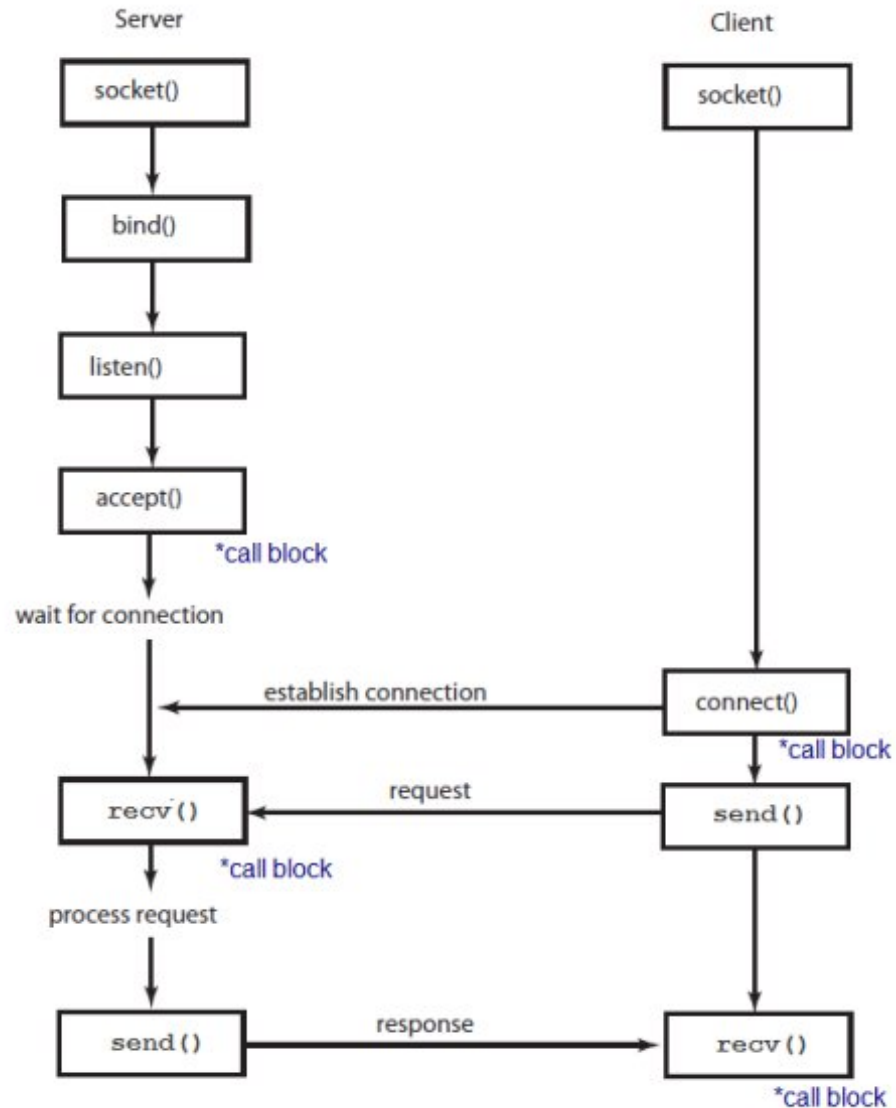
```
#include <iostream>
#include <thread>
#include <windows.h>
using namespace std;
int Flag = 0;

void TextEingabe() {
    char Eingabe;
    cout << "Dieser Thread wartete nur auf Eingabe... s für Ende\n";
    cin >> Eingabe;
    if (Eingabe == 's' || Eingabe == 'S') Flag = 1;
}

void SomeTextOutput() {
    for (int z = 1; z < 1000; z++) {
        cout << z << "\tDieserThread erzeugt einfach Textausgabe\n";
        if (Flag == 0) Sleep(1); else break;
    }
}

int main() {
    thread t1(TextEingabe); // thread #1
    thread t2(SomeTextOutput); // thread #2
    t1.join(); t2.join(); return 0;}
```

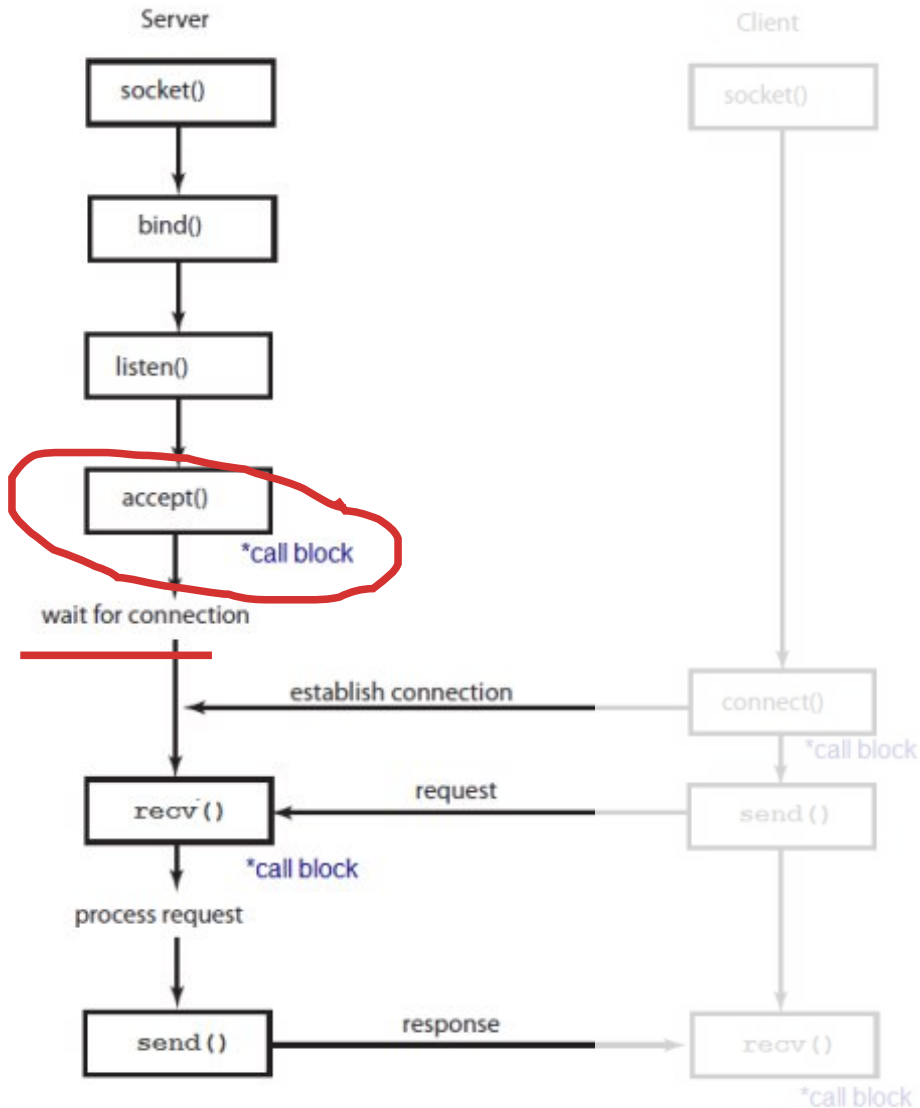
Zwei weitere sehr nützliche Beispiele...



Wie kann ein Programm sowohl Server als auch Client gleichzeitig sein?

Wie kann ein Programm Server mehrere Clients gleichzeitig haben?

Beispiel P2P



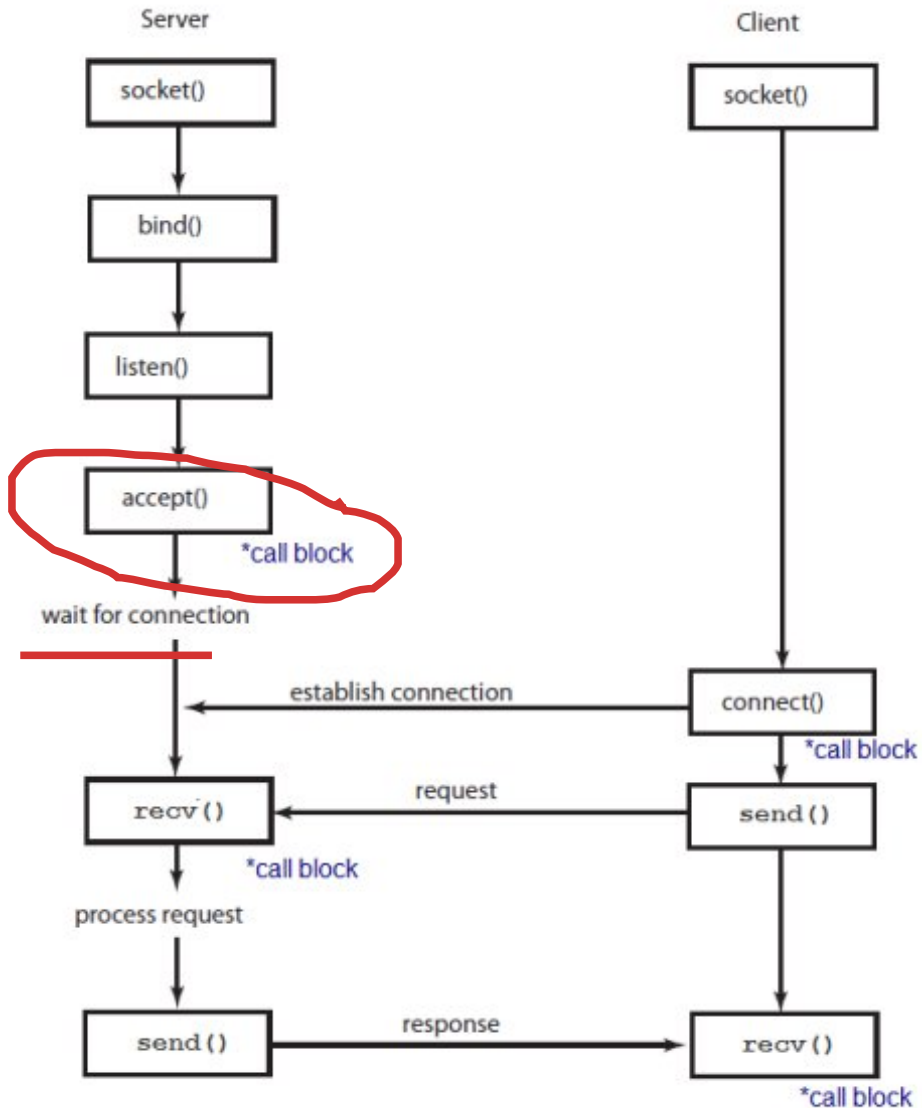
```
int ServerAcceptingConnection(SOCKET* ConnectSocket);
```

```
void Test(SOCKET* ServerSock, SOCKET* ClientSock, sockaddr_in* Client_Set)
{
    thread t1(ServerAcceptingConnection, ServerSock); // server thread localhost
}
```

```
t1.join();
cout << "MAIN : Thread 1 finished... \n";
}
```

```
int ServerAcceptingConnection(SOCKET* ConnectSocket) {
    // Accepting a Client Connection
    int clientSocket = accept(*ConnectSocket, nullptr, nullptr);
    [...]
}
```

Beispiel P2P



```
int ServerAcceptingConnection(SOCKET* ConnectSocket);

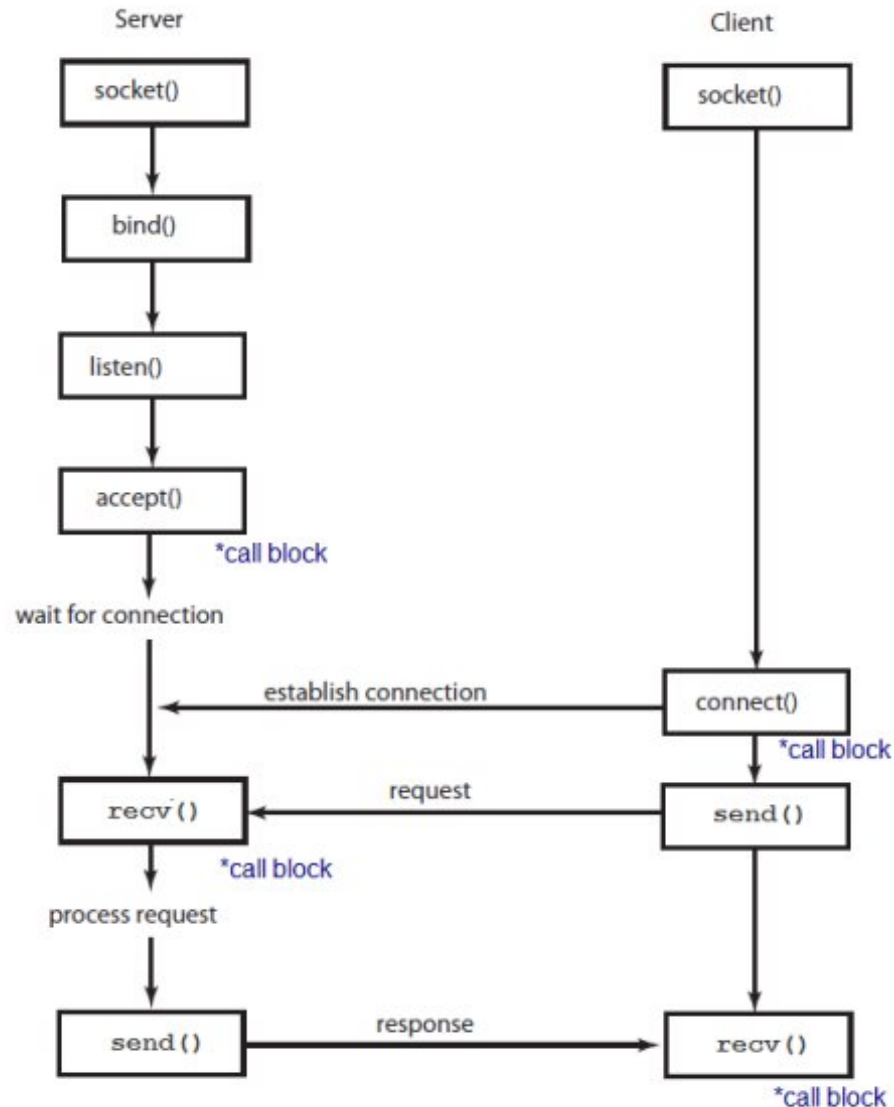
void Test(SOCKET* ServerSock, SOCKET* ClientSock, sockaddr_in* Client_Set)
{
    thread t1(ServerAcceptingConnection, ServerSock); // server thread localhost

    Connect2Server(ClientSock, Client_Set); // Connect to localhost server

    t1.join();
    cout << "MAIN : Thread 1 finished... \n";
}

int ServerAcceptingConnection(SOCKET* ConnectSocket) {
    // Accepting a Client Connection
    int clientSocket = accept(*ConnectSocket, nullptr, nullptr);
    [...]
}
```


Beispiel mehrere Clients



```
int ServerAcceptingConnection(SOCKET* ConnectSocket);

void Test(SOCKET* ServerSock, SOCKET* ClientSock, sockaddr_in* Client_Set)
{
    thread t1(ServerAcceptingConnection, ServerSock); // server thread #1 localhost
    thread t2(ServerAcceptingConnection, ServerSock); // server thread #2 localhost

    Connect2Server(ClientSock, Client_Set); // Connect to localhost server

    // Start second client to connect to the server... (now to thread t2)
    SOCKET* ClientSocket2 = FireUpSocket();
    Connect2Server(ClientSocket2, Client_Set); // Connect to thread #2 of
                                              // localhost server

    t2.join(); // pauses until thread finishes
    cout << "MAIN : Thread 2 finished... \n";
    t1.join();
    cout << "MAIN : Thread 1 finished... \n";
}

int ServerAcceptingConnection(SOCKET* ConnectSocket) {
    // Accepting a Client Connection
    int clientSocket = accept(*ConnectSocket, nullptr, nullptr);
    [...]
}
```

